Programming Exercise 1

- Image Rotation

  Recall that transformation matrices can be used to rotate vectors. For example the rotation matrix will rotate a vector 45 degrees *Counter Clock Wise(CCW).* Points may be thought of as vectors from the origin, so we could equivalently say that it will rotate a point by 45 degrees *CCW* about the origin.

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \qquad A = \begin{bmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix} = \begin{bmatrix} \mathbf{cos(45°)} & \mathbf{-sin(45°)} \\ \mathbf{sin(45°)} & \mathbf{cos(45°)} \end{bmatrix}$$

a. Rotate an image by 45 degrees as below example using forward method which calculates destination coordinates from original coordinates.

b. Rotate an image by 45 degrees as below example using backward method.

c. If you rounded coordinates on (b.), you used nearest neighbor interpolation method. Do rotate an image using bilinear interpolation method.

(https://en.wikipedia.org/wiki/Bilinear_interpolation)



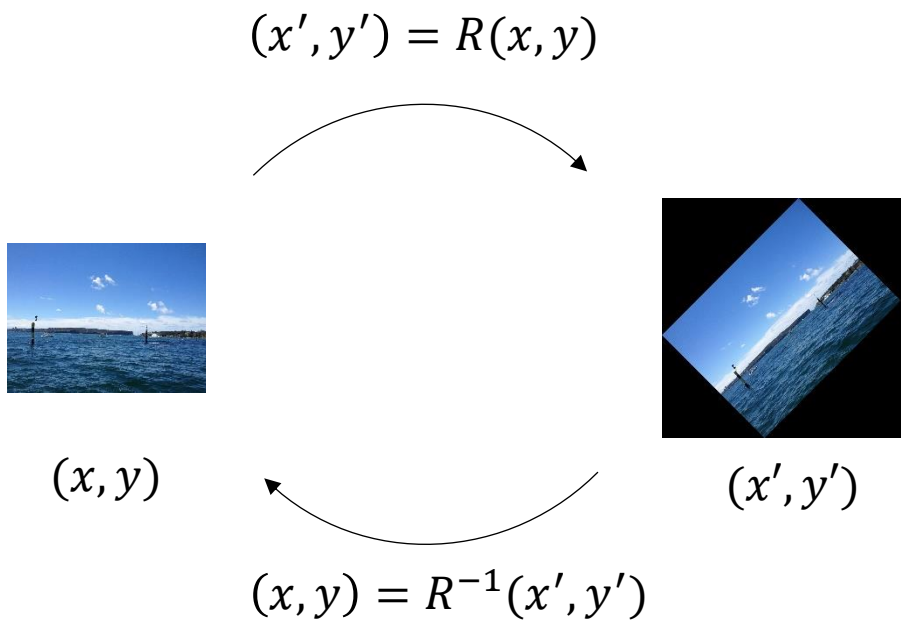
Input



Output

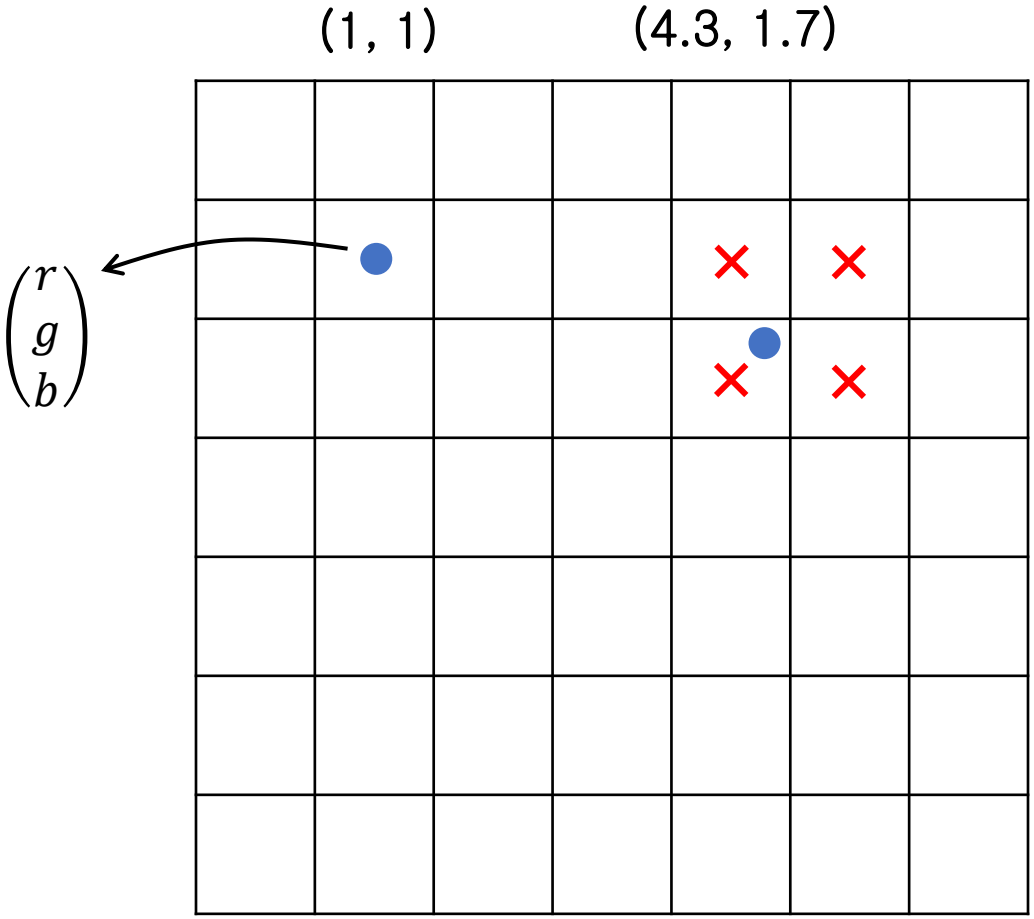
Computer Vision Programming Exercise 1

- Forward method: mapping with forward computation
  - For each (x, y), calculate (x', y') by $(x', y') = R(x, y)$
  - map $f(x, y)$ to $f(x', y')$
- Backward method: Mapping with backward computation
  - For each (x', y'), calculate (x, y) by $(x, y) = R^{-1}(x', y')$
  - map $f(x, y)$ to $f(x', y')$



Computer Vision Programming Exercise 1

- What if $(x, y) = R^{-1}(x', y')$ is not integer

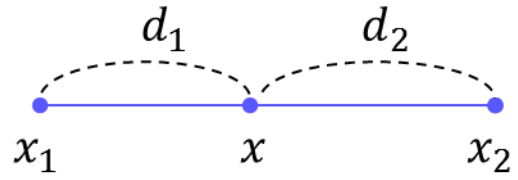


Computer Vision Programming Exercise 1

- Linear interpolation

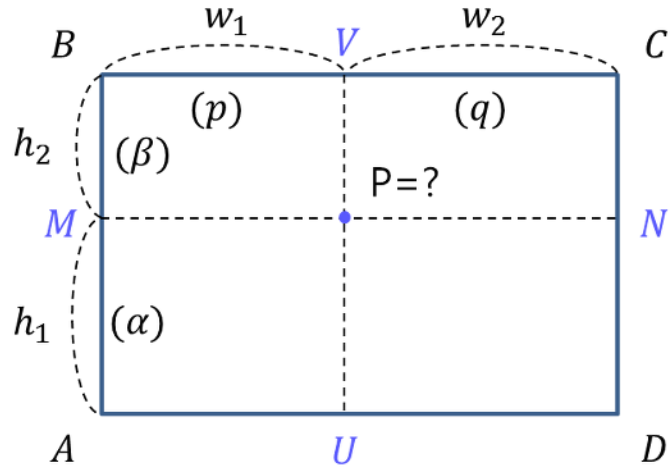
α = d1/(d1+d2), β = d2/(d1+d2)

$$f(x)=\beta f(x_1)+\alpha f(x_2)$$



- Bilinear interpolation

$$P=q(\beta A+\alpha B)+p(\beta D+\alpha C)$$
$$=q\beta A+q\alpha B+p\beta D+p\alpha C$$



How to do Exercise

1. Open the project file with Visual studio.

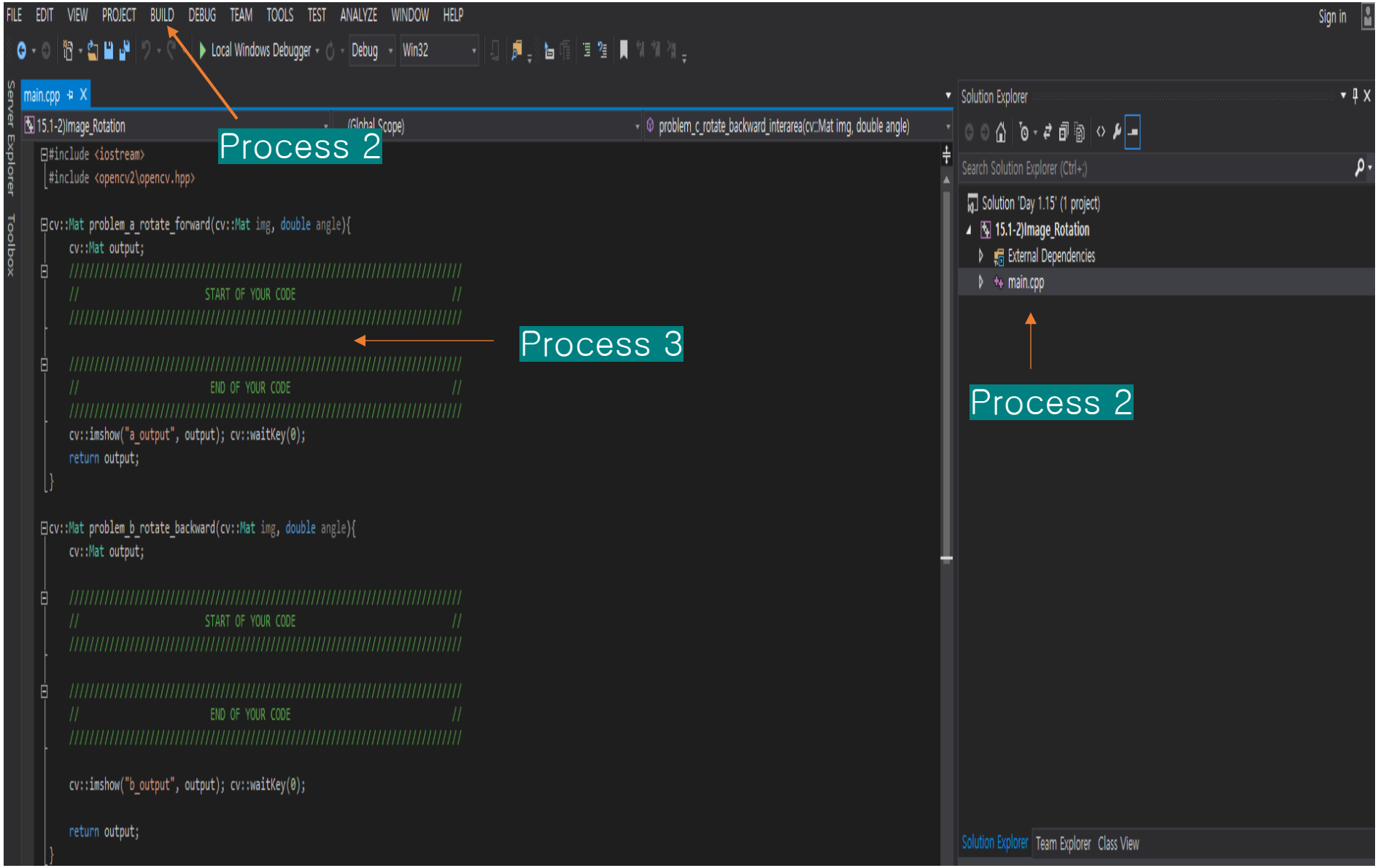
2. Click the build toolbar at the top, and click ‘build solution’ (“Image rotation”).

3. Write your code in “main.cpp” file in “Image_Rotation” Solution.

4. The above processes are outlined on slide 6.

5. Environment Settings are written on slide 7.

How to do Exercise




```
#include <iostream>
#include <opencv2\opencv.hpp>

cv::Mat problem_a_rotate_forward(cv::Mat img, double angle){
    cv::Mat output;
    ////////////////////////////////////////////////////////////////
    //                     START OF YOUR CODE                     //
    ////////////////////////////////////////////////////////////////


    ////////////////////////////////////////////////////////////////
    //                      END OF YOUR CODE                      //
    ////////////////////////////////////////////////////////////////
    cv::imshow("a_output", output); cv::waitKey(0);
    return output;
}

cv::Mat problem_b_rotate_backward(cv::Mat img, double angle){
    cv::Mat output;

    ////////////////////////////////////////////////////////////////
    //                     START OF YOUR CODE                     //
    ////////////////////////////////////////////////////////////////


    ////////////////////////////////////////////////////////////////
    //                      END OF YOUR CODE                      //
    ////////////////////////////////////////////////////////////////

    cv::imshow("b_output", output); cv::waitKey(0);

    return output;
}
```

Environment Setup

- Do the Exercise in Visual Studio

(Depending on VS version, you may need to handle different errors.)

- Language : C / C++
- Required packages and libraries : OpenCV

(OpenCV packages are included in project folder.)

Example Results

- Input Image



A. Forward Method



B. Backward Method



C. Backward and Interpolation



Grading policy

- Due to many students and with only one TA, we need to simplify the grading process
  - Failure in compilation/build : 0 point
  - Failure in passing copy check : 0 point
  - Failure in generating results of A, B, and C : −30 points deduction for each